Assignments Guidelines

1. Introduction

- The projects are personal for each student that expressed their interest in the development of the assignments. They will have a common part and a unique part.
- The description of the projects, together with some hints on how to develop it, are listed in this file SSDS-Assignments-24_25-s000000.pdf that is sent to you via email.
- The deadline for handling the projects is at the end of the exam session, i.e. **February 28th**
- The projects must be written in VHDL, and you need to use Vivado to make the projects (exceptions can be considered)

2. Deliverables

For us to evaluate your work, you will have to send us via mail by the deadline a .zip folder containing the compressed Vivado project folder. it must include:

- All the .vhd files that describe your project
- All the other files that are necessary to make the project in Vivado (e.g. memory initialization files, test vectors, waveform configurations)
- A brief report where you describe your project:

2.1 Written Report

The report will be used to evaluate your methodology in the making of the project, hence try to be as clear as possible and put all the information you think is useful to explain your choices.

Mandatory sections are:

- A list of all the files used in the project and a brief description of their purpose.
- A description of the FSM (e.g. states and their function, state diagram)
- A brief description of the testbench you implemented and the cases you test with the expected results.

3. Evaluation

The evaluation of the assignment will be made according to these criteria:

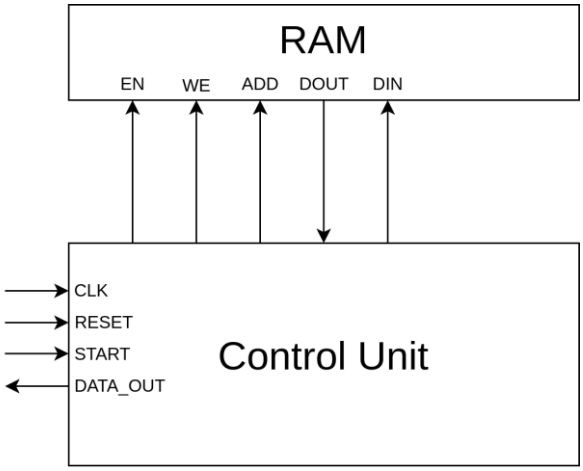
- Exhaustivity and clarity of the written report;
- Correctness of the project according to the assignment;
- If all files can be simulated in a correct way.

4. Assignment

Design a circuit that **counts the occurrences of a number in a list.** The input data array must be read sequentially from the main memory element (a RAM), and according to whether there has been an occurrence of the target number, a counter must be increased.

The architecture will be composed as follows:

- A RAM.
- The Control Unit, that reads data from the memory and performs the operations



4.1 Operations

The first address of the RAM (ADD 0) stores the number for which the number of occurrences must be evaluated. At each iteration the address is increased, the next data is read, the occurrence evaluated and eventually the counter increased by 1.

The input data for this assignment will be **8-bit** wide (all positive), stored in a **32x8** memory (32 strings of **8-bit** each). Your circuit will read the number at each address, evaluate the occurrence of the target number and output the result.

4.2 RAM

To insert the RAM element in your design, import inside your Vivado project the RAM.vhd file that is provided within the assignment. It has been written by taking as a starting point the VHDL template *No change Mode* inside the built-in Vivado library, available in the Software at

Language Templates>VHDL>Synthesis Constructs>Coding Examples>RAM>BlockRAM>Single Port

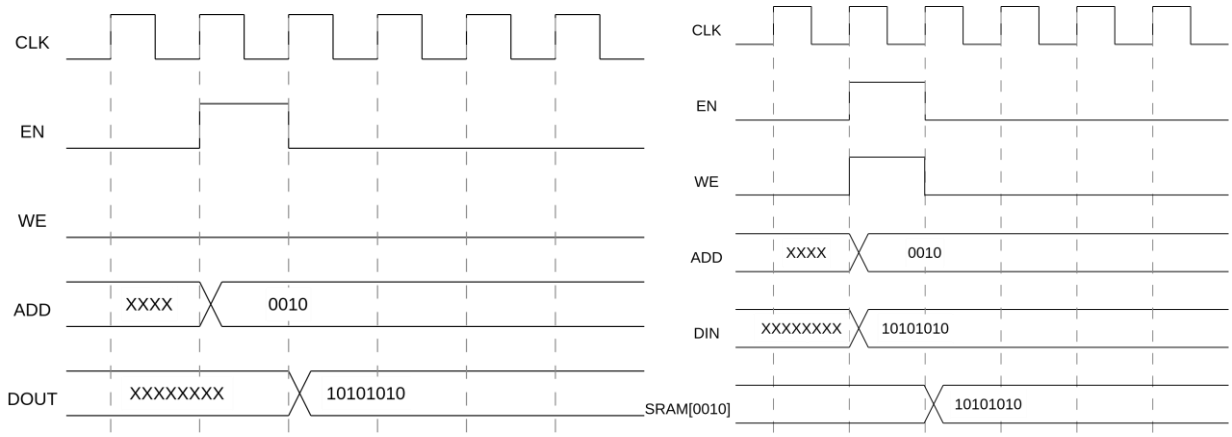
The RAM model described in the template follows a simple working principle typical of SRAM (timing diagram below):

- Some GENERIC constants that allow the parametrization of the memory (
  - RAM_WIDTH: length of the word stored in the RAM
  - RAM_DEPTH: number of words stored in the RAM (a power of 2)
  - RAM_ADD, nr. of bits of the address (= $\log_2$ (*RAM_DEPTH*))

- INIT_FILE: name of the memory initialization file
- Data Ports (One port for address and **two** for data, input for write operations and output for read operations)
- Control signals (EN and WE for controlling R/W operations, CLK)

Modify all the necessary parameters inside the RAM.vhd file according to the RAM specifications.
You can also access the template and modify the provided .vhd file to add additional functions to your RAM (e.g. by adding an output register that is used to increase the performances)

4.3 RAM READ/WRITE Timing diagram



Read and write operation on a RAM: the signal EN controls the access to the memory, which outputs the data at the requested address if WE = 0, or writes to it at the specified address if WE = 1

4.4 RAM initialization file

To facilitate the initialization of the memory with the data that will be read, it is possible to add a .mem file to your Vivado project with the purpose of starting operation with already full RAM. The initialization of the RAM is made by a specific function init_from_file_or_zeroes that stores the data as specified in the file or initializes it to all zeros.
For example, in a 16x8 RAM, with a memory.mem file structured like this:

11001100
11111111
10101010
11110000

At the beginning of your simulation the RAM will be initialized as follows:

ADD 0000: 11001100
ADD 0001: 11111111
ADD 0010: 10101010
ADD 0011: 11110000
ADD from 0100 to 1111: 11110000 (it should take the last data in your file: I suggest putting it to all zeros, in the case you don't want to initialize all the memory)

Add a memory initialization file in your project for your testbench.

4.5 Control Unit

Design an FSM-based system (synchronous reset) that interacts with the main memory: this unit is in charge of managing the read operations for moving data from the RAM by using the control signals (EN, WE, …).
The input signal *start* signals the CU to start the operations by assuming value '**1**' for one clock cycle.
The operations must be as follows:

- Data0 is read from the RAM at address 0 and saved in a register
- Data1 is read from address 1, if Data1 == Data0 the internal counter should be increased, otherwise it keeps its value
- Data2 is read from address 2, if Data2 == Data0 the internal counter should be increased, otherwise it keeps its value

And so on.
Whenever the CU starts the operations, it must start from address 0 and read until address 31.
The output signal DATA_OUT should always represent the value of the counter, i.e. the number of the occurrences of the target number

5 Testbench

- Add a testbench to your project to test the design in relevant cases
- Create and use a memory initialization file that allows you to initialize the RAM
- It would be better to also save the waveform configuration file (and include it in the project) if there are some signal you consider important for testing purposes